Лабораторная работа № 2. Подготовка данных и оконные функции PostgreSQL

Каждый студент выполняет **один назначенный вариант**. Данные находятся в одноимённой папке архива `sql_window_datasets.zip`. Файлы `.txt` имеют формат UTF-8 TSV: первая строка содержит имена столбцов, разделитель — табуляция. В папке варианта находятся также `SCHEMA.md` с перечнем полей и связей и `schema.sql` для создания итоговых таблиц PostgreSQL.

1. Описание вариантов

Вариант	Предметная область и данные	Основная таблица фактов	Особенности, которые нужно учитывать
01	**Маркетплейс.** `regions`, `categories`, `sellers`, `customers`, `products`, `orders`, `order_items`. Покупатель оформляет заказ, а каждая позиция относится к товару конкретного продавца.	`orders` — 10 000 заказов	Стоимость заказа вычисляется по `order_items.quantity * order_items.unit_price`. Возможны гостевые заказы. Статус заказа нужно нормализовать.
02	**Доставка еды.** `districts`, `restaurants`, `couriers`, `customers`, `courier_shifts`, `orders`, `deliveries`. Заказ оформляется в ресторане; успешный заказ имеет запись о доставке.	`orders` — 10 000 заказов	Отменённые заказы могут не иметь доставки. Смена и доставка связаны с одним курьером. Время доставки считают по временным меткам выбранного этапа, явно указав этот выбор.
03	**Стриминговый сервис.** `genres`, `users`, `titles`, `viewing_sessions`, `views`. Один сеанс может содержать несколько просмотров.	`viewing_sessions` — 10 000 сеансов	Для активности используйте фактически просмотренное время `watched_seconds`, а не полную длительность произведения.
04	**Онлайн-образование.** `study_groups`, `students`, `courses`, `modules`, `topics`, `tests`, `test_attempts`, `topic_progress`. Студент может проходить тест несколько раз.	`test_attempts` — 10 000 попыток	Балл за тест — лучшая попытка студента; итоговый балл — сумма лучших результатов по тестам. Успешной считать попытку от 60 баллов.
05	**Городской транспорт.** `routes`, `stops`, `vehicles`, `route_stops`, `trips`,	`trips` — 10 000 рейсов	Пассажиропоток считают по посадкам. Загрузку после остановки получают

Вариант	Предметная область и данные	Основная таблица фактов	Особенности, которые нужно учитывать
	`stop_events`. Каждый рейс проходит несколько остановок.		накопительной суммой посадок минус высадки и сравнивают с вместимостью машины.
06	**Гостиницы.** `cities`, `hotels`, `rooms`, `guests`, `bookings`. Бронирование связывает гостя с номером на период проживания.	`bookings` — 10 000 бронирований	Для сопоставления цен используйте цену за ночь: `total_amount / число ночей`. Загрузка отеля — занятые номеро-ночи относительно доступных номеро-ночей за выбранный период.
07	**Склад и поставки.** `warehouses`, `products`, `suppliers`, `deliveries`, `delivery_items`, `stock_movements`. Поставка содержит позиции, складские движения отражают приход и расход.	`deliveries` — 10 000 поставок	Остаток рассчитывают по паре «склад — товар». Для прихода количество берут со знаком плюс, для расхода — со знаком минус. Отменённые поставки не формируют приход.
08	**Мобильное приложение.** `cohorts`, `users`, `screens`, `features`, `sessions`, `events`. В сеансе хранится последовательность действий пользователя.	`sessions` — 10 000 сеансов	Не всякое событие связано с экраном или функцией: соответствующие поля могут быть пустыми. События с одинаковым временем упорядочивают по `event_id`.
09	**Спортивные соревнования.** `seasons`, `teams`, `players`, `matches`, `player_match_stats`. Матч связывает две команды; статистика игроков относится к конкретному матчу.	`matches` — 10 000 матчей	Для расчёта результатов команд разверните матч в две строки: одна на каждую команду. За победу начисляйте 3 очка, за ничью 1, за поражение 0.
10	**Энергопотребление зданий.** `buildings`, `meters`, `meter_readings`, `meter_events`. Счётчик хранит накопительные показания.	`meter_readings` — 10 080 показаний	Между соседними показаниями два часа. Событие `reset` означает сброс счётчика; разность через сброс нельзя считать расходом. Часть зданий не имеет счётчика.

Число строк дочерних таблиц фактов может превышать 10 000: например, одному рейсу соответствуют несколько остановок. Небольшие классификаторы, такие как жанры и сезоны, содержат меньше 100

строк. Во всех вариантах сохранены связи по ключам, но имеются пробелы и различия регистра в названиях и статусах, пустые необязательные поля, редкие выбросы и одинаковые временные метки.

2. Загрузка и очистка данных

2.1. Сырой слой

Для **каждого** `.txt`-файла создайте временную таблицу `raw_<имя_таблицы>`. Имена и порядок столбцов должны совпадать с заголовком файла. На сыром этапе объявите все столбцы как `TEXT`: исходное значение должно сохраниться до проверки и преобразования.

Загрузите файлы командой `COPY FROM`. Например, для варианта 01:

```
CREATE TEMP TABLE raw_orders (
    order_id TEXT,
    customer_id TEXT,
    ordered_at TEXT,
    status TEXT,
    sales_channel TEXT
);

COPY raw_orders
FROM 'C:/путь/variant_01_marketplace/orders.txt'
WITH (FORMAT csv, DELIMITER E'\t', HEADER true,
      NULL '', ENCODING 'UTF8');
```

Обратите внимание: `COPY FROM` обычно загружает строки в таблицу. Временное представление само по себе не хранит строки; PostgreSQL допускает загрузку в представление лишь при специальном триггере `INSTEAD OF INSERT`. Поэтому в этой работе `COPY` направляется во **временные таблицы**, а **временные представления** создаются для последовательных этапов очистки. Путь в `COPY` должен быть доступен серверу PostgreSQL. Если файл есть только на компьютере студента, в `psql` можно использовать клиентскую команду `\copy` с теми же параметрами формата. Все временные объекты действуют только в текущем подключении. Документация PostgreSQL: COPY, CREATE VIEW.

2.2. Последовательные временные представления

Создайте как минимум **два этапа очистки** для каждой таблицы, где есть загрязнение:

1. `v_<таблица>_normalized` — удалите лишние пробелы по краям текстовых значений, приведите статус к единому регистру, преобразуйте пустые необязательные значения в `NULL`. Первичный ключ и внешний ключ нельзя исправлять произвольно.
2. `v_<таблица>_typed` — после проверки приведите идентификаторы и целочисленные величины к `INTEGER`, суммы к `NUMERIC`, даты и время к нужным типам. Добавьте вычисляемые признаки некорректных или подозрительных записей там, где это нужно для вашего варианта.

Прежде чем применять приведение типов, выполните запросы профилирования. Для каждой таблицы проверьте количество строк, пустые обязательные поля, повторяющиеся первичные ключи и ссылки на отсутствующие справочные записи. Для измерений проверьте минимум, максимум и подозрительные

выбросы. Для времени проверьте невозможный порядок событий: например, доставку до создания заказа или окончание просмотра до его начала.

Результат проверки определяет следующее изменение представления. Повторите проверку после изменения. Не удаляйте необычное значение только потому, что оно далеко от среднего: сохраните обоснование решения **оставить, исправить или исключить** запись. Если запись исключена, отразите её идентификатор и причину в контрольном запросе или отдельной таблице отклонений.

2.3. Итоговые типизированные таблицы

После очистки выполните `schema.sql` своего варианта. Он создаёт постоянные таблицы в отдельной схеме. Перенесите в них строки из итоговых временных представлений явными командами `INSERT INTO ... (столбцы) SELECT ...`. Сначала заполните справочники, затем зависимые таблицы. Все запросы с оконными функциями выполняйте **по итоговым типизированным таблицам или представлениям, построенным на них**.

Зафиксируйте сверку по каждой таблице:

```
строк в сыром файле = загружено в итоговую таблицу + обоснованно отклонено
```

Проверьте, что типы столбцов соответствуют `schema.sql`, внешние ключи ссылаются на существующие строки, а повторная проверка не обнаруживает необработанных статусов и пробелов по краям названий. Сохраните контрольные запросы в SQL-файле.

3. Запросы с оконными функциями

Выполните задания **только своего варианта**. Нумерация одинакова для всех вариантов:

- `R1–R6` — ранжирование и распределение;
- `N1–N5` — соседние строки и значения в окне;
- `A1–A5` — обычные агрегаты с `OVER`.

В каждом запросе покажите идентификатор объекта, столбцы группировки, исходный показатель и вычисленное оконной функцией значение. Укажите в комментарии, что означает одна строка результата. Если показатель сначала нужно рассчитать по множеству строк, подготовьте его в CTE или подзапросе, затем примените оконную функцию.

Для `row_number()`, `lag()`, `lead()` и выбора конкретного по счёту события задайте однозначный порядок: **время, затем идентификатор события**. Для `rank()` и `dense_rank()` не добавляйте уникальный идентификатор в порядок ранжирования, если он уничтожит равенство показателей. Покажите хотя бы один случай равенства и объясните различие мест с пропусками и без пропусков. Когда показатель почти всегда уникален, используйте обоснованное округление или дискретный показатель и явно опишите это в комментарии.

Для `last_value()` и `nth_value()` задайте рамку окна явно, если нужен результат по всей группе. Если в группе меньше `n` строк, `nth_value()` вернёт `NULL`. Для `sum()`, `count()`, `avg()`, `min()`, `max()` и `array_agg()` требуется **оконное применение с `OVER`**; предварительная обычная агрегация допускается для подготовки показателя. Документация PostgreSQL: оконные функции.

Вариант 01. Маркетплейс

№	Запрос	Подсказка
R1	Найдите последний заказ каждого зарегистрированного покупателя.	`row_number()`
R2	Ранжируйте продавцов по выручке внутри региона; равная выручка должна давать место с пропуском следующего номера.	`rank()`
R3	Ранжируйте товары внутри категории по числу проданных единиц без пропуска мест.	`dense_rank()`
R4	Определите относительное положение выручки продавца среди продавцов его региона.	`percent_rank()`
R5	Для каждого товара покажите долю товаров его категории с текущей ценой не выше данной.	`cume_dist()`
R6	Разделите покупателей на четыре группы по общей сумме завершённых покупок.	`ntile(4)`
N1	Покажите сумму предыдущего заказа рядом с каждым заказом покупателя.	`lag()`
N2	Покажите дату следующего заказа покупателя.	`lead()`
N3	Покажите канал первого заказа рядом с каждым заказом покупателя.	`first_value()`
N4	Покажите статус последнего заказа рядом с каждым заказом покупателя.	`last_value()`
N5	Покажите сумму второго заказа покупателя; для покупателей с одним заказом оставьте `NULL`.	`nth_value(..., 2)`
A1	Покажите накопительную выручку продавца по завершённым заказам.	`sum() over`
A2	Покажите число заказов покупателя на дату каждого его заказа.	`count() over`
A3	Покажите среднюю стоимость последних пяти заказов покупателя.	`avg() over`
A4	Покажите минимальную и максимальную стоимость заказа покупателя.	`min()`, `max()` с `OVER`
A5	Для каждой позиции заказа покажите массив товаров этого заказа.	`array_agg() over`

Стоимость заказа рассчитывайте по позициям, а выручку продавца — по принадлежащим ему товарам. Для выручки используйте только завершённые заказы; в задачах о последовательности заказов учитывайте все статусы, если формулировка не уточняет иное.

Вариант 02. Доставка еды

№	Запрос	Подсказка
R1	Найдите последнюю доставку каждого курьера.	`row_number()`

№	Запрос	Подсказка
R2	Ранжируйте рестораны района по количеству завершённых заказов с пропусками мест при равенстве.	`rank()`
R3	Ранжируйте курьеров по числу доставок без пропуска мест.	`dense_rank()`
R4	Определите положение среднего времени доставки ресторана среди ресторанов района.	`percent_rank()`
R5	Для доставки покажите долю доставок того же ресторана с длительностью не больше данной.	`cume_dist()`
R6	Разделите курьеров на четыре группы по числу доставок.	`ntile(4)`
N1	Измерьте время между текущей и предыдущей завершённой доставкой курьера.	`lag()`
N2	Покажите время следующего назначения курьера.	`lead()`
N3	Покажите ресторан первого заказа клиента рядом с каждым его заказом.	`first_value()`
N4	Покажите ресторан последнего заказа клиента рядом с каждым его заказом.	`last_value()`
N5	Покажите время второй доставки курьера в каждой смене.	`nth_value(..., 2)`
A1	Рассчитайте накопленную выручку ресторана в течение каждого дня.	`sum() over`
A2	Покажите количество доставок курьера в каждой смене.	`count() over`
A3	Рассчитайте среднюю длительность последних десяти доставок курьера.	`avg() over`
A4	Покажите минимальную и максимальную длительность доставки курьера.	`min()`, `max()` с `OVER`
A5	Составьте упорядоченный массив ресторанов, из которых курьер забирал заказы в смену.	`array_agg() over`

Для длительности доставки используйте интервал от `picked_up_at` до `delivered_at`. Для запроса N2 упорядочивайте назначения по `assigned_at`, а для запросов о выполненных доставках — по `delivered_at`.

Вариант 03. Стриминговый сервис

№	Запрос	Подсказка
R1	Найдите последний просмотр каждого пользователя.	`row_number()`
R2	Ранжируйте фильмы внутри жанра по числу просмотров с пропусками мест.	`rank()`
R3	Ранжируйте сериалы по суммарному времени просмотра без пропуска мест.	`dense_rank()`

№	Запрос	Подсказка
R4	Определите положение фильма по числу просмотров внутри жанра.	`percent_rank()`
R5	Для произведения покажите долю произведений его жанра с длительностью не больше данной.	`cume_dist()`
R6	Разделите пользователей на четыре группы по суммарному времени просмотра.	`ntile(4)`
N1	Найдите интервал между соседними просмотрами пользователя.	`lag()`
N2	Для просмотра сериала покажите время следующего просмотра этого же сериала пользователем.	`lead()`
N3	Покажите жанр первого просмотренного произведения пользователя.	`first_value()`
N4	Покажите жанр последнего просмотренного произведения пользователя.	`last_value()`
N5	Покажите название второго просмотренного произведения пользователя.	`nth_value(..., 2)`
A1	Рассчитайте накопленное время просмотра пользователя в пределах месяца.	`sum() over`
A2	Покажите число просмотров в сеансе рядом с каждым просмотром.	`count() over`
A3	Рассчитайте среднюю длительность последних пяти просмотров пользователя.	`avg() over`
A4	Покажите первый и последний моменты просмотра пользователя за день.	`min()`, `max()` с `OVER`
A5	Составьте упорядоченный массив произведений, просмотренных за сеанс.	`array_agg() over`

Сортируйте просмотры по `views.started_at, views.view_id`. Продолжительность фактического просмотра берите из `watched_seconds`.

Вариант 04. Онлайн-образование

№	Запрос	Подсказка
R1	Найдите последнюю попытку студента по каждому тесту.	`row_number()`
R2	Ранжируйте студентов внутри группы по итоговому баллу с пропусками мест.	`rank()`
R3	Ранжируйте студентов той же группы без пропуска мест при равном балле.	`dense_rank()`
R4	Определите положение итогового балла студента в группе.	`percent_rank()`
R5	Покажите долю студентов группы с итоговым баллом не выше данного.	`cume_dist()`

№	Запрос	Подсказка
R6	Разделите студентов группы на четыре категории по итоговому баллу.	`ntile(4)`
N1	Сравните балл попытки с предыдущей попыткой студента по тому же тесту.	`lag()`
N2	Для неудачной попытки покажите дату следующей попытки того же теста.	`lead()`
N3	Покажите балл первой попытки студента по тесту.	`first_value()`
N4	Покажите балл последней попытки студента по тесту.	`last_value()`
N5	Покажите балл третьей попытки студента по тесту.	`nth_value(..., 3)`
A1	Рассчитайте накопленную сумму баллов попыток студента внутри модуля.	`sum() over`
A2	Покажите число попыток студента по тесту.	`count() over`
A3	Рассчитайте средний балл последних трёх попыток студента по тесту.	`avg() over`
A4	Покажите худший и лучший балл студента по тесту.	`min()`, `max()` с `OVER`
A5	Составьте упорядоченный массив тем, пройденных студентом в курсе.	`array_agg() over`

Для R2–R6 сначала найдите лучший балл каждого студента по каждому тесту, затем сложите результаты. Для N2 вычислите `lead()` **до** отбора неудачных попыток, иначе следующий результат может быть потерян.

Вариант 05. Городской транспорт

№	Запрос	Подсказка
R1	Пронумеруйте рейсы каждого маршрута за день по времени отправления.	`row_number()`
R2	Ранжируйте маршруты по числу вошедших пассажиров с пропусками мест.	`rank()`
R3	Ранжируйте остановки по числу вошедших пассажиров без пропусков мест.	`dense_rank()`
R4	Определите относительное положение задержки рейса среди рейсов его маршрута.	`percent_rank()`
R5	Покажите долю рейсов маршрута с задержкой не больше задержки данного рейса.	`cume_dist()`
R6	Разделите рейсы на четыре группы по максимальной загрузке относительно вместимости машины.	`ntile(4)`

№	Запрос	Подсказка
N1	Вычислите интервал после предыдущего отправления на том же маршруте в тот же день.	`lag()`
N2	Покажите время следующего отправления маршрута в тот же день.	`lead()`
N3	Покажите время первого отправления маршрута за день.	`first_value()`
N4	Покажите время последнего отправления маршрута за день.	`last_value()`
N5	Покажите время третьего рейса маршрута за день.	`nth_value(..., 3)`
A1	Рассчитайте накопленное число вошедших пассажиров маршрута за день.	`sum() over`
A2	Покажите число рейсов маршрута за день.	`count() over`
A3	Рассчитайте среднюю задержку последних десяти рейсов маршрута.	`avg() over`
A4	Покажите минимальную и максимальную задержку рейса маршрута.	`min()`, `max()` с `OVER`
A5	Составьте массив остановок рейса в порядке следования.	`array_agg() over`

Задержку определяйте по `actual_departure - scheduled_departure`. Для R6 сначала получите накопленное количество пассажиров после каждой остановки, затем найдите максимум для рейса.

Вариант 06. Гостиницы

№	Запрос	Подсказка
R1	Найдите последнее бронирование каждого гостя.	`row_number()`
R2	Ранжируйте отели города по загрузке за период, охваченный данными.	`rank()`
R3	Ранжируйте отели города по выручке без пропуска мест.	`dense_rank()`
R4	Определите положение цены за ночь среди бронирований города.	`percent_rank()`
R5	Покажите долю бронирований города с ценой за ночь не выше текущей.	`cume_dist()`
R6	Разделите гостей на четыре группы по расходам на проживание.	`ntile(4)`
N1	Найдите интервал между выездом и следующим заездом гостя.	`lag()`
N2	Покажите дату следующего бронирования гостя.	`lead()`
N3	Покажите отель первого проживания гостя.	`first_value()`
N4	Покажите отель последнего проживания гостя.	`last_value()`
N5	Покажите стоимость второго проживания гостя.	`nth_value(..., 2)`

№	Запрос	Подсказка
A1	Рассчитайте накопленные расходы гостя за календарный год.	`sum() over`
A2	Покажите число проживаний гостя.	`count() over`
A3	Рассчитайте среднюю цену последних трёх бронирований гостя.	`avg() over`
A4	Покажите минимальную и максимальную цену за ночь среди бронирований отеля.	`min()`, `max()` с `OVER`
A5	Составьте массив отелей в порядке проживаний гостя.	`array_agg() over`

Для расчёта проживания и выручки используйте `LOWER(BTRIM(status)) = 'completed'`. В R2 опишите период и знаменатель: число номеров отеля × число дней периода.

Вариант 07. Склад и поставки

№	Запрос	Подсказка
R1	Найдите последнее движение каждого товара на каждом складе.	`row_number()`
R2	Ранжируйте поставщиков по суммарному объёму полученных товаров.	`rank()`
R3	Ранжируйте товары по оборачиваемости без пропуска мест: расход за период / средний положительный остаток.	`dense_rank()`
R4	Определите положение фактического срока поставки среди полученных поставок.	`percent_rank()`
R5	Покажите долю полученных поставок, у которых фактический срок не больше текущего.	`cume_dist()`
R6	Разделите товары на четыре группы по суммарному объёму расхода.	`ntile(4)`
N1	Сравните объём поступления товара с объёмом его предыдущей поставки на тот же склад.	`lag()`
N2	Покажите дату следующей запланированной поставки товара на склад.	`lead()`
N3	Покажите первый зафиксированный остаток товара на складе.	`first_value()`
N4	Покажите последний зафиксированный остаток товара на складе.	`last_value()`
N5	Покажите объём третьей полученной поставки товара на склад.	`nth_value(..., 3)`
A1	Рассчитайте остаток после каждого движения товара на складе.	`sum() over`
A2	Покажите число полученных поставок товара на склад.	`count() over`
A3	Рассчитайте средний объём последних пяти поставок товара на склад.	`avg() over`
A4	Покажите минимум и максимум рассчитанного остатка за месяц.	`min()`, `max()` с `OVER`

№	Запрос	Подсказка
A5	Составьте массив поставщиков товара на склад в порядке поступления.	`array_agg() over`

Для R4–R5 под сроком понимается отклонение `received_at - planned_at`; оно может быть отрицательным при раннем поступлении. Для R3 укажите, как обрабатываете нулевой средний остаток. В задачах об остатке сортируйте по `occurred_at, movement_id`.

Вариант 08. Мобильное приложение

№	Запрос	Подсказка
R1	Найдите последнее событие каждого пользователя.	`row_number()`
R2	Ранжируйте функции приложения по числу использований.	`rank()`
R3	Ранжируйте экраны по числу открытий без пропуска мест.	`dense_rank()`
R4	Определите положение активности пользователя среди пользователей его когорты.	`percent_rank()`
R5	Покажите долю сеансов пользователя с длительностью не больше текущей.	`cume_dist()`
R6	Разделите пользователей на четыре группы по числу событий за выбранную календарную неделю.	`ntile(4)`
N1	Найдите интервал с предыдущего события пользователя в том же сеансе.	`lag()`
N2	Для открытия экрана покажите следующее действие в том же сеансе.	`lead()`
N3	Покажите первое действие пользователя в каждом сеансе.	`first_value()`
N4	Покажите последнее действие пользователя в каждом сеансе.	`last_value()`
N5	Покажите третье действие пользователя в каждом сеансе.	`nth_value(..., 3)`
A1	Рассчитайте накопленную длительность сеансов пользователя.	`sum() over`
A2	Покажите число событий в сеансе рядом с каждым событием.	`count() over`
A3	Рассчитайте среднюю длительность последних семи сеансов пользователя.	`avg() over`
A4	Покажите минимальную и максимальную длительность сеанса пользователя.	`min()`, `max()` с `OVER`
A5	Составьте массив экранов, открытых в сеансе, в порядке событий.	`array_agg() over`

В R4 активность определяйте как число событий за весь наблюдаемый период. Для R6 укажите выбранную неделю и включите пользователей с нулём событий. В A5 исключите события без `screen_id`.

Вариант 09. Спортивные соревнования

№	Запрос	Подсказка
R1	Пронумеруйте матчи команды в пределах сезона.	`row_number()`
R2	Рассчитайте места команд по набранным очкам после каждого тура.	`rank()`
R3	Ранжируйте игроков по забитым голам без пропуска мест.	`dense_rank()`
R4	Определите положение игрока по числу голов среди игроков той же позиции.	`percent_rank()`
R5	Покажите долю игроков с числом голов не выше текущего.	`cume_dist()`
R6	Разделите игроков на четыре группы по суммарному игровому времени.	`ntile(4)`
N1	Сравните число голов команды с числом голов в её предыдущем матче сезона.	`lag()`
N2	Покажите следующего соперника команды в сезоне.	`lead()`
N3	Покажите результат первого матча команды в сезоне.	`first_value()`
N4	Покажите результат последнего матча команды в сезоне.	`last_value()`
N5	Покажите результат пятого матча команды в сезоне.	`nth_value(..., 5)`
A1	Рассчитайте накопленные очки команды после каждого матча сезона.	`sum() over`
A2	Покажите число сыгранных матчей команды в сезоне.	`count() over`
A3	Рассчитайте среднее число очков за последние пять матчей команды.	`avg() over`
A4	Покажите минимум и максимум голов команды за матч в сезоне.	`min()`, `max()` с `OVER`
A5	Составьте массив соперников команды в порядке матчей сезона.	`array_agg() over`

Сначала подготовьте представление или CTE «команда — матч — сезон — тур — очки». В R2 место вычисляют **после тура**, поэтому сначала суммируют очки команды до конца тура, затем сравнивают команды одного сезона и тура. Суммарные голы игроков берут из `player_match_stats`.

Вариант 10. Энергопотребление зданий

№	Запрос	Подсказка
R1	Найдите последнее показание каждого счётчика.	`row_number()`
R2	Ранжируйте здания по потреблению за наблюдаемый период.	`rank()`
R3	Ранжируйте здания по максимальному расходу за двухчасовой интервал без пропуска мест.	`dense_rank()`

№	Запрос	Подсказка
R4	Определите положение суточного потребления здания среди наблюдаемых суток этого здания.	`percent_rank()`
R5	Покажите долю суток здания с потреблением не выше текущего.	`cume_dist()`
R6	Разделите здания со счётчиками на четыре группы по суммарному расходу за наблюдаемый период.	`ntile(4)`
N1	Вычислите разницу между текущим и предыдущим показанием счётчика.	`lag()`
N2	Покажите время следующего показания счётчика.	`lead()`
N3	Покажите первое показание счётчика в наблюдаемом месяце.	`first_value()`
N4	Покажите последнее показание счётчика в наблюдаемом месяце.	`last_value()`
N5	Покажите третье показание счётчика за сутки.	`nth_value(..., 3)`
A1	Рассчитайте накопленный расход счётчика за наблюдаемый месяц.	`sum() over`
A2	Покажите число показаний счётчика за сутки.	`count() over`
A3	Рассчитайте среднее суточное потребление за последние семь наблюдаемых суток.	`avg() over`
A4	Покажите минимальный и максимальный расход за двухчасовой интервал.	`min()`, `max()` с `OVER`
A5	Для каждого счётчика составьте массив временных меток его максимальных расходов за сутки.	`array_agg() over`

Расход для первой строки счётчика неизвестен. При событии `reset` начинайте новую последовательность показаний и **не прибавляйте отрицательную разницу** к расходу. Измерения идут с шагом два часа; если рассчитываете среднюю мощность за час, поделите расход интервала на два и явно укажите единицы. Данные охватывают примерно семь суток на счётчик, поэтому задания R2 и R6 используют наблюдаемый период, а не год.

4. Что сохранить и сдать

Передайте папку своего варианта с SQL-файлами:

Файл	Содержание
`00_raw_load.sql`	Создание всех временных сырьевых таблиц и `COPY` для всех `.txt`-файлов варианта.
`01_cleaning.sql`	Профилирование, последовательные временные представления, правила очистки и проверки после каждой итерации.
`02_publish.sql`	Создание итоговых типизированных таблиц, перенос очищенных данных в порядке зависимостей.

Файл	Содержание
`03_ranking.sql`	Шесть запросов `R1–R6`, каждый с комментарием о гранулярности и порядке сортировки.
`04_neighbors.sql`	Пять запросов `N1–N5`.
`05_aggregates.sql`	Пять запросов `A1–A5`. В `A4` должны присутствовать и `min()`, и `max()`.
`06_checks.sql`	Сверка числа строк, ключей, типов, связей и несколько контрольных проверок результатов оконных запросов.

Разделяйте запросы комментариями с номером задания. Файлы `00_raw_load.sql` и `01_cleaning.sql` запускайте в **одном подключении**, поскольку временные объекты пропадут после окончания сессии. В начале каждого файла укажите схему своего варианта и порядок запуска. Для `COPY` оставьте понятное место для абсолютного пути к данным.

В `06_checks.sql` обязательно включите проверку одного случая равных рангов, первого и последнего события группы, группы с недостаточным числом строк для `nth_value()` и сравнение накопительной суммы с обычной итоговой суммой по той же группе. Если какой-то такой случай отсутствует в данных вашего варианта, создайте небольшой отдельный контрольный набор данных в SQL-файле и покажите результат на нём.

5. Вопросы при защите

На защите студент запускает выбранные преподавателем запросы, показывает промежуточное представление очистки и отвечает на вопросы по собственному SQL. Возможные вопросы:

1. Что означает одна строка результата вашего запроса? Какие столбцы задают `PARTITION BY` и почему?
2. Чем `row_number()` отличается от `rank()` и `dense_rank()` при равных значениях?
3. Почему `rank()` может пропускать места, а `dense_rank()` — нет?
4. Как вычисляются `percent_rank()` и `cume_dist()`? Что они вернут при равных значениях и группе из одной строки?
5. Как `ntile(4)` распределяет строки, если число строк не делится на четыре?
6. Что вернут `lag()` и `lead()` на границах группы? Как указать значение по умолчанию?
7. Почему одинаковые временные метки требуют дополнительного столбца в `ORDER BY` для выбора соседней строки?
8. Что входит в рамку окна по умолчанию при наличии `ORDER BY`? Почему это важно для `last_value()` и `nth_value()`?
9. Чем накопительная сумма отличается от суммы по всей группе? Как явно задать обе рамки?
10. Чем `sum() over (...)` отличается от `GROUP BY` по количеству строк результата?
11. Как ведут себя `count(*) over (...)` и `count(column) over (...)` при `NULL`?
12. Как порядок строк влияет на результат `array_agg() over (...)`?
13. Почему файл сначала загружен в текстовую временную таблицу, а не сразу в типизированную итоговую?
14. Почему `COPY FROM` в этой работе направлен в таблицу, а временные представления используются для очистки?
15. Как доказать, что после очистки не потеряны строки и не нарушены связи?

Преподаватель может изменить в контрольном запросе одну границу рамки, столбец сортировки или условие фильтра и попросить объяснить, как изменится результат.